In [2]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call d
rive.mount("/content/drive", force_remount=True).
```

In [3]:
```
pip install --upgrade keras
```

```
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages
(3.13.2)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packag
es (from keras) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from keras) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
(from keras) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
(from keras) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages
(from keras) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-package
s (from keras) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-pack
ages (from keras) (0.5.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pack
ages (from keras) (26.0)
Requirement already satisfied: typing-extensions>=4.6.0 in /usr/local/lib/pytho
n3.12/dist-packages (from optree->keras) (4.15.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python
3.12/dist-packages (from rich->keras) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python
3.12/dist-packages (from rich->keras) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pac
kages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)
```

In [4]:
```
from tensorflow import keras
```

In [5]:
```
pip show tensorflow
```

```
Name: tensorflow
Version: 2.19.0
Summary: TensorFlow is an open source machine learning framework for everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: /usr/local/lib/python3.12/dist-packages
Requires: absl-py, astunparse, flatbuffers, gast, google-pasta, grpcio, h5py, k
eras, libclang, ml-dtypes, numpy, opt-einsum, packaging, protobuf, requests, se
tuptools, six, tensorboard, termcolor, typing-extensions, wrapt
Required-by: dopamine_rl, tensorflow-text, tensorflow_decision_forests, tf_kera
s
```

In [6]:
```
pip show keras
```

```
Name: keras
Version: 3.13.2
Summary: Multi-backend Keras
Home-page:
Author:
Author-email: Keras team <keras-users@googlegroups.com>
License: Apache License 2.0
Location: /usr/local/lib/python3.12/dist-packages
Requires: absl-py, h5py, ml-dtypes, namex, numpy, optree, packaging, rich
Required-by: keras-hub, tensorflow
```

In [7]:
```
pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-pac
kages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dis
t-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/d
ist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.1
2/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/loca
l/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.1
2/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/di
st-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/d
ist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pack
ages (from tensorflow) (26.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.2
1.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-pack
ages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.1
2/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pac
kages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-pa
ckages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/di
st-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/pytho
n3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.1
2/dist-packages (from tensorflow) (1.78.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.1
2/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-p
ackages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.1
2/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-p
ackages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python
3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/
dist-packages (from astunparse>=1.6.0->tensorflow) (0.46.3)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
(from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
(from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-package
s (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pytho
```

```
n3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-p
ackages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/
dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/
dist-packages (from requests<3,>=2.21.0->tensorflow) (2026.2.25)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dis
t-packages (from tensorboard~=2.19.0->tensorflow) (3.10.2)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/lo
cal/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dis
t-packages (from tensorboard~=2.19.0->tensorflow) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/d
ist-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python
3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python
3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pac
kages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)
```

In [8]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.utils import array_to_img, img_to_array, load_img
# If you intend to use `image_dataset_from_directory`, please uncomment and en
# from tensorflow.keras.utils import image_dataset_from_directory

datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')
```

In [9]:
```
import tensorflow.keras as keras
```

In [14]:
```
img = load_img(r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/HAPPY/kids-10.j
```

In [15]:
```
img
```

Out[15]:

In [17]:
```
import os
```

```
x = img_to_array(img)
x = x.reshape((1,)+x.shape)

# Define the directory to save augmented images
save_directory = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/HAPPY/augment

# Create the directory if it doesn't exist
if not os.path.exists(save_directory):
    os.makedirs(save_directory)

i = 0
for batch in datagen.flow(x, batch_size=1, save_to_dir=save_directory, save_pr
  i += 1
  if i > 30:
    break
```

In [19]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.utils import image_dataset_from_directory
from keras.utils import array_to_img,img_to_array,load_img

datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='reflect'
)
img = load_img(r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/HAPPY/kids-10.j
```

In [20]:
```
img
```

Out[20]:

In [23]:
```
x = img_to_array(img)
x = x.reshape((1,)+x.shape)

i = 0
for batch in datagen.flow(x,batch_size=1,
                          save_to_dir=r"/content/drive/MyDrive/CNN HAPPY SAD/T
    i += 1
    if i > 30:
        break
```

In [24]:
```
keras.applications.ResNet50(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/resnet/resnet50_weights_tf_dim_ordering_tf_kernels.h5
102967424/102967424 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

Out[24]:
```
<Functional name=resnet50, built=True>
```

In [25]:
```
from keras.utils import array_to_img,img_to_array,load_img
img = load_img(r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/HAPPY/kids-10.j
img
```

Out[25]:

In [28]:
```
import numpy as np
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input,
from tensorflow.keras.preprocessing import image

# Load The ResNet-50 model pre-trained on ImageNet data
model = ResNet50(weights='imagenet')

# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/HAPPY/kids-10.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0) # ResNet50 expects a batch dimen
img_array = preprocess_input(img_array)
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predicted:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i+1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/da
ta/imagenet_class_index.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Predicted:
1: pajama (0.32)
2: whippet (0.07)
3: Bouvier_des_Flandres (0.06)

Top Prediction Class Index: 697
```

In [33]:
```python
import numpy as np
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decoc
from tensorflow.keras.preprocessing import image
# Load the VGG16 model pre-trained on ImageNet data
model = VGG16(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/HAPPY/kids-10.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Make predictions
predictions = model.predict(img_array)
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]

print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 770ms/step
Predictions:
1: fur_coat (0.53)
2: poncho (0.04)
3: cloak (0.04)

Top Prediction Class Index: 568
```

In [34]:
```python
from keras.utils import array_to_img,img_to_array,load_img
img = load_img(r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/alone
```

In [35]:
```python
img
```

Out[35]:

In [38]:

```
import numpy as np
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decoc
from tensorflow.keras.preprocessing import image
# Load the VGG16 model pre-trained on ImageNet data
model = VGG16(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/alone-1.j
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Make predictions
predictions = model.predict(img_array)
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 763ms/step
Predictions:
1: lakeside (0.13)
2: monitor (0.07)
3: corn (0.06)

Top Prediction Class Index: 975
```

In [39]:

```
from keras.utils import array_to_img,img_to_array,load_img
img = load_img(r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.j
```

In [40]:

```
img
```

Out[40]:

In [42]:

```
import numpy as np
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input, decoc
from tensorflow.keras.preprocessing import image
# Load the VGG19 model pre-trained on ImageNet data
model = VGG19(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Make predictions
predictions = model.predict(img_array)
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/vgg19/vgg19_weights_tf_dim_ordering_tf_kernels.h5
574710816/574710816 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.mak
e_predict_function.<locals>.one_step_on_data_distributed at 0x7d9c4099de40> tri
ggered tf.function retracing. Tracing is expensive and the excessive number of
tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) pa
ssing tensors with different shapes, (3) passing Python objects instead of tens
ors. For (1), please define your @tf.function outside of the loop. For (2), @t
f.function has reduce_retracing=True option that can avoid unnecessary retracin
g. For (3), please refer to https://www.tensorflow.org/guide/function#controlli
ng_retracing and https://www.tensorflow.org/api_docs/python/tf/function for  mo
re details.
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 815ms/step
Predictions:
1: tiger_cat (0.48)
2: tiger (0.21)
3: fountain (0.11)

Top Prediction Class Index: 282
```

In [44]:
```
import numpy as np
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input, decod
from tensorflow.keras.preprocessing import image
# Load the VGG19 model pre-trained on ImageNet data
model = VGG19(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Make predictions
predictions = model.predict(img_array)
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=8)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

```
WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.mak
e_predict_function.<locals>.one_step_on_data_distributed at 0x7d9c2f50bba0> tri
ggered tf.function retracing. Tracing is expensive and the excessive number of
tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) pa
ssing tensors with different shapes, (3) passing Python objects instead of tens
ors. For (1), please define your @tf.function outside of the loop. For (2), @t
f.function has reduce_retracing=True option that can avoid unnecessary retracin
g. For (3), please refer to https://www.tensorflow.org/guide/function#controlli
ng_retracing and https://www.tensorflow.org/api_docs/python/tf/function for  mo
re details.
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 837ms/step
Predictions:
1: tiger_cat (0.48)
2: tiger (0.21)
3: fountain (0.11)
4: tabby (0.06)
5: lynx (0.05)
6: Egyptian_cat (0.03)
7: jaguar (0.01)
8: bittern (0.01)

Top Prediction Class Index: 282
```

In [46]:
```
import numpy as np
from tensorflow.keras.applications.resnet_v2 import ResNet50V2, \
    preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
# Load the ResNet50V2 model pre-trained on ImageNet data
model = ResNet50V2(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
```

```
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Make predictions
predictions = model.predict(img_array)
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=10)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels.h5
102869336/102869336 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
Predictions:
1: fountain (0.43)
2: tiger (0.35)
3: tiger_cat (0.14)
4: lynx (0.04)
5: jaguar (0.02)
6: tabby (0.01)
7: Egyptian_cat (0.00)
8: cougar (0.00)
9: brown_bear (0.00)
10: Bouvier_des_Flandres (0.00)

Top Prediction Class Index: 562
```

In [48]:
```
from keras.utils import array_to_img,img_to_array,load_img
img = load_img(r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.j
img
```

Out[48]:

In [50]:
```
import numpy as np
import time
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input,
    decode_predictions
from tensorflow.keras.preprocessing import image
# Load the ResNet-50 model pre-trained on ImageNet data
model = ResNet50(weights='imagenet')
# Load and preprocess the input image
```

```
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))

img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
Predictions:
1: lynx (0.56)
2: fountain (0.15)
3: tabby (0.07)
4: Egyptian_cat (0.06)
5: tiger (0.03)

Top Prediction Class Index: 287
Inference Time: 3746.99 ms
Size (MB): 97.80 MB
Parameters: 25636712
Depth: 177
```

In [52]:

```
import numpy as np
import time
from tensorflow.keras.applications.resnet_v2 import ResNet50V2, \
    preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
# Load the ResNet50V2 model pre-trained on ImageNet data
model = ResNet50V2(weights='imagenet')
# Load and preprocess the input image
```

```
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
Predictions:
1: fountain (0.43)
2: tiger (0.35)
3: tiger_cat (0.14)
4: lynx (0.04)
5: jaguar (0.02)

Top Prediction Class Index: 562
Inference Time: 3307.21 ms
Size (MB): 97.71 MB
Parameters: 25613800
Depth: 192
```

In [54]:

```
import numpy as np
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input, \
    decode_predictions
from tensorflow.keras.preprocessing import image
import time
# Load the VGG19 model pre-trained on ImageNet data
model = VGG19(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
```

```
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Predictions:
1: tiger_cat (0.48)
2: tiger (0.21)
3: fountain (0.11)
4: tabby (0.06)
5: lynx (0.05)

Top Prediction Class Index: 282
Inference Time: 1359.25 ms
Size (MB): 548.05 MB
Parameters: 143667240
Depth: 26
```

In [56]:

```
import numpy as np
import time
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decoc
from tensorflow.keras.preprocessing import image

# Load the VGG16 model pre-trained on ImageNet data
model = VGG16(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
```

```
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 811ms/step
Predictions:
1: fountain (0.99)
2: tiger_cat (0.00)
3: tiger (0.00)
4: tabby (0.00)
5: lynx (0.00)

Top Prediction Class Index: 562
Inference Time: 852.19 ms
Size (MB): 527.79 MB
Parameters: 138357544
Depth: 23
```

In [58]:

```
import numpy as np
import time
from tensorflow.keras.applications import Xception
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.xception import preprocess_input, decode_pr

# Load the Xception model pre-trained on ImageNet data
model = Xception(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(299, 299)) # Xception requires inp
```

```
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/xception/xception_weights_tf_dim_ordering_tf_kernels.h5
91884032/91884032 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 23s 23s/step
Predictions:
1: tiger_cat (0.27)
2: lynx (0.25)
3: tabby (0.07)
4: Egyptian_cat (0.05)
5: tiger (0.01)

Top Prediction Class Index: 282
Inference Time: 41002.29 ms
Size (MB): 87.40 MB
Parameters: 22910480
Depth: 134
```

In [60]:

```
import numpy as np
import time
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.inception_v3 import preprocess_input, decoc
# Load the InceptionV3 model pre-trained on ImageNet data
model = InceptionV3(weights='imagenet')
```

```
# Load and preprocess the input image

img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(299, 299)) # Xception requires inp
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/inception_v3/inception_v3_weights_tf_dim_ordering_tf_kernels.h5
96112376/96112376 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 9s 9s/step
Predictions:
1: tiger_cat (0.39)
2: Egyptian_cat (0.23)
3: tabby (0.18)
4: lynx (0.10)
5: fountain (0.00)

Top Prediction Class Index: 282
Inference Time: 8688.87 ms
Size (MB): 90.99 MB
Parameters: 23851784
Depth: 313
```

In [62]:

```
import numpy as np
import time
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing import image
```

```
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decod
# Load the MobileNetV2 model pre-trained on ImageNet data
model = MobileNetV2(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5
14536120/14536120 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 13s 13s/step
Predictions:
1: lynx (0.42)
2: tiger_cat (0.06)
3: coyote (0.05)
4: fountain (0.04)
5: dhole (0.02)

Top Prediction Class Index: 287
Inference Time: 12723.50 ms
Size (MB): 13.50 MB
Parameters: 3538984
Depth: 156
```

In [68]:

```
import numpy as np
import time
```

```
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.applications.densenet import preprocess_input, \
    decode_predictions
from tensorflow.keras.preprocessing import image
# Load the DenseNet121 model pre-trained on ImageNet data
model = DenseNet121(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)

img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 11s 11s/step
Predictions:
1: lynx (0.97)
2: tiger_cat (0.01)
3: tabby (0.01)
4: Egyptian_cat (0.01)
5: snow_leopard (0.00)

Top Prediction Class Index: 287
Inference Time: 10979.97 ms
Size (MB): 30.76 MB
Parameters: 8062504
Depth: 429
```

In [70]:
```
import numpy as np
```

```python
import time
from tensorflow.keras.applications import NASNetMobile
from tensorflow.keras.applications.nasnet import preprocess_input, \
    decode_predictions
from tensorflow.keras.preprocessing import image
# Load the NASNetMobile model pre-trained on ImageNet data
model = NASNetMobile(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/nasnet/NASNet-mobile.h5
24227760/24227760 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 37s 37s/step
Predictions:
1: Egyptian_cat (0.31)
2: tiger_cat (0.25)
3: tabby (0.23)
4: lynx (0.03)
5: jaguar (0.01)

Top Prediction Class Index: 285
Inference Time: 37051.82 ms
Size (MB): 20.32 MB
Parameters: 5326716
Depth: 771
```

In [72]:

```python
import numpy as np
import time
from tensorflow.keras.applications import NASNetLarge
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.nasnet import preprocess_input, \
    decode_predictions
# Load the NASNetLarge model pre-trained on ImageNet data
model = NASNetLarge(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(331, 331)) # NASNetLarge requires
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
```

```
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/nasnet/NASNet-large.h5
359748576/359748576 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 50s 50s/step
Predictions:
1: tiger_cat (0.45)
2: tabby (0.41)
3: Egyptian_cat (0.05)
4: lynx (0.01)
5: tiger (0.00)

Top Prediction Class Index: 282
Inference Time: 49641.27 ms
Size (MB): 339.32 MB
Parameters: 88949818
Depth: 1041
```

In [74]:

```
import numpy as np
import time
from tensorflow.keras.applications import EfficientNetV2B0
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.efficientnet_v2 import preprocess_input, \
    decode_predictions
# Load the EfficientNetV2B0 model pre-trained on ImageNet data
model = EfficientNetV2B0(weights='imagenet')
# Load and preprocess the input image
img_path = r"/content/drive/MyDrive/CNN HAPPY SAD/TRAINING/NOT HAPPY/cat.jpg"
img = image.load_img(img_path, target_size=(224, 224)) # EfficientNetV2B0 defa
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)
# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()
# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")
# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
```

```python
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applicati
ons/efficientnet_v2/efficientnetv2-b0.h5
29403144/29403144 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 13s 13s/step
Predictions:
1: lynx (0.39)
2: tiger_cat (0.13)
3: tabby (0.08)
4: Egyptian_cat (0.08)
5: fountain (0.04)

Top Prediction Class Index: 287
Inference Time: 20520.70 ms
Size (MB): 27.47 MB
Parameters: 7200312
Depth: 273
```